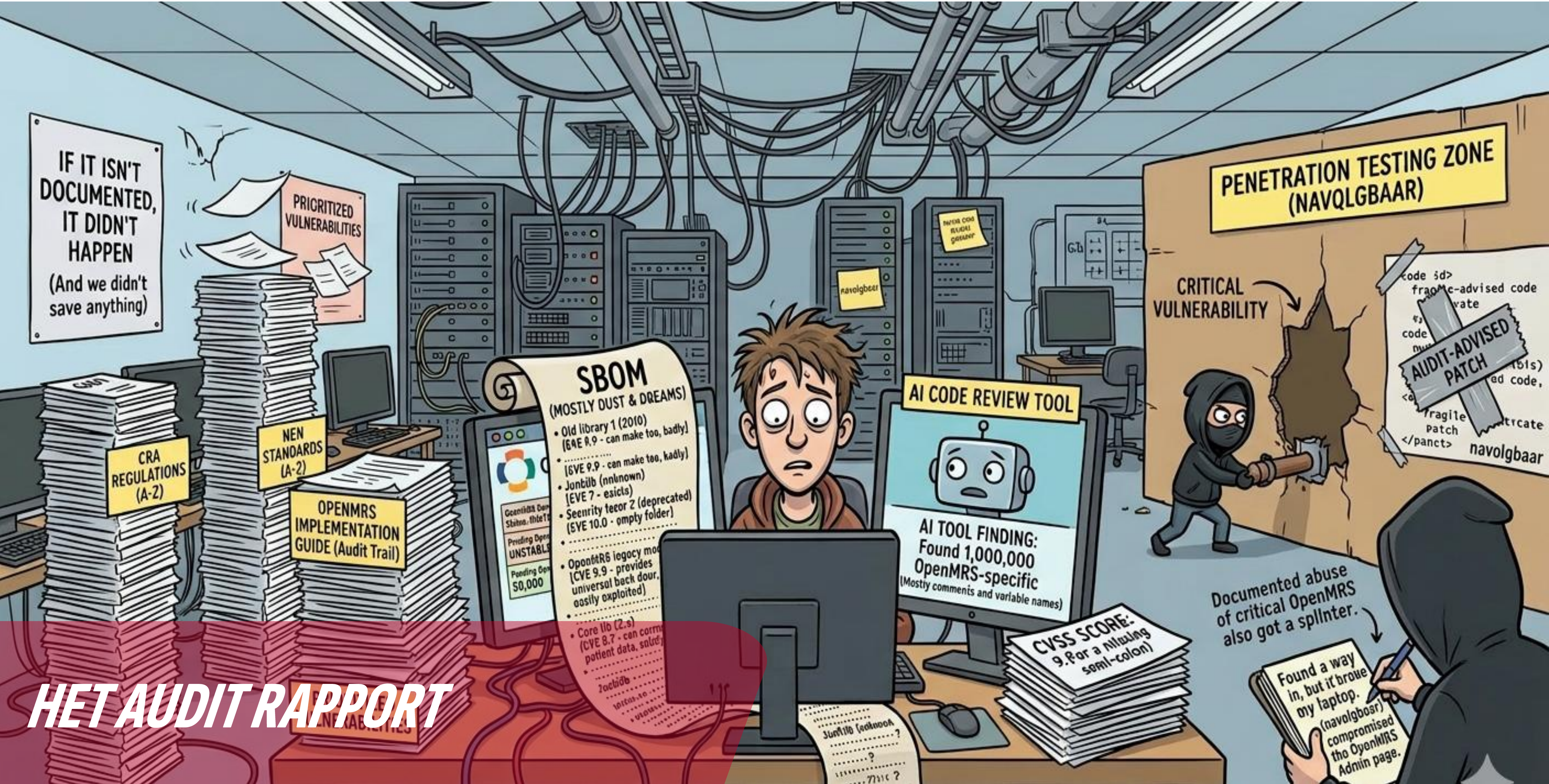




HET AUDIT RAPPORT

WAAR STAAN WE

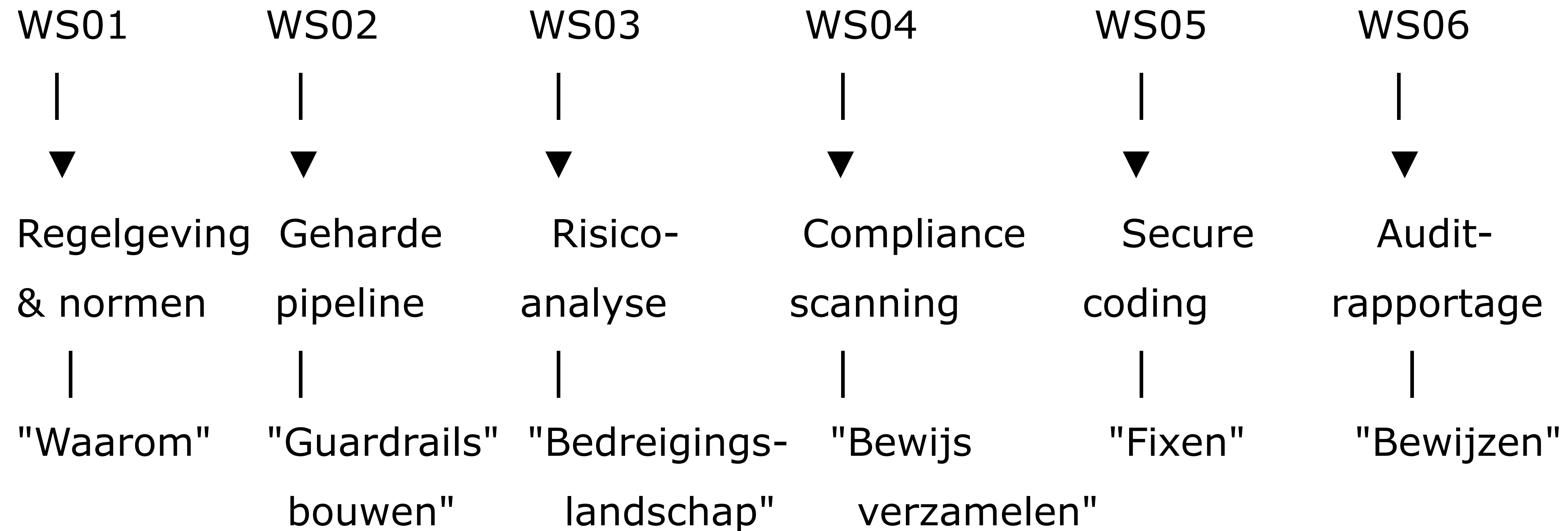
Workshop	Thema	Deliverable
WS01	Wet- & regelgeving, NEN-7510, audit mindset	Begrippenkader
WS02	Hardening CI/CD pipeline (CodeQL, Dependabot, SBOM)	Beveiligde pipeline
WS03	Healthcare Risk Assessment (risicomatrix, bow-tie)	Risicorapport
WS04	Compliance Scanning (CVE/CVSS, Snyk, security backlog)	Scanrapport + backlog
WS05	Secure Coding & Privacy by Design	Logging + tests
WS06	Formeel auditrapport	Definitief auditrapport

LEERDOELEN

Na deze workshop kun je...

- ✓ Een volledig evidence-based auditrapport schrijven voor een softwaremodule.
- ✓ Technische bevindingen vertalen naar managementtaal en onderscheid maken tussen de behoeften van management en technisch team.
- ✓ De relatie uitleggen tussen CRA-verplichtingen en de corresponderende NEN-7510:2024-2 controls.
- ✓ Een responsible disclosure procedure uitvoeren.
- ✓ Een traceability matrix opstellen die elke NEN-7510-claim koppelt aan een concrete maatregel en een verifieerbaar bewijs-artefact.

6 workshops, 1 traject



6 workshops, 1 traject

Workshop	Artefact	Functie in het auditrapport
WS01	Normenanalyse, logging-checklist	Scope & normenkader
WS02	Branch protection rules, CodeQL workflow, SBOM	Technische bevindingen & bijlagen
WS03	Risicomatrix, bow-tie diagram	Risico-analyse sectie
WS04	Snyk-rapport, security backlog, CVE-lijst	Bevindingen & SBOM sectie
WS05	Secure code PR's, unit tests, logging code	Traceability matrix (vóór/na)
WS06	Volledig auditrapport	Opzet naar eindproduct

Audit?

Definitie: Een audit is een systematische, onafhankelijke beoordeling van een organisatie, systeem of proces om vast te stellen of het voldoet aan vastgestelde criteria.

Drie vormen:

- **Interne audit:** uitgevoerd door de eigen organisatie (bijv. security team)
- **Externe audit:** uitgevoerd door een onafhankelijke partij (bijv. certificeringsinstantie)
- **Zelfevaluatie:** zoals jullie opdracht: aantonen dat je aan de norm voldoet

De auditor stelt altijd drie vragen:

- Wat heb je afgesproken? (beleid, normen, controls)
- Wat heb je gedaan? (maatregelen, technische implementaties)
- Hoe bewijs je dat? (logs, screenshots, rapporten, test-output)

NEN-7510: 5.35:

- *Onafhankelijke beoordeling van informatiebeveiliging — De organisatie moet periodiek onafhankelijk laten beoordelen of het informatiebeveiligingsbeleid en de implementatie ervan adequaat zijn.*

Terugblik WS 1 | Wetten / richtlijnen / normen

Regelgeving & normen: het fundament

Begrip	Aard	Voorbeeld	Afdwingbaar?
Wet	Nationaal recht	Wet BIG, AVG (nationale implementatie)	Ja, via rechter
Verordening	EU-recht, rechtstreeks	AVG (GDPR), CRA	Ja, direct
Richtlijn	EU-recht, via implementatie	NIS2 (→ Cyberbeveiligingswet)	Na omzetting
Norm	Vrijwillige standaard	NEN-7510:2024, ISO 27001	Contractueel

De audit mindset: wie, wat, waar, wanneer, waarom — de 5 W's van logging zijn terug in elke bevinding die gedocumenteerd wordt.

Terugblik WS 2 | Guardrails

Hardened CI/CD: fundament

- Branch protection rules: nooit direct naar main pushen
- SAST: statische analyse bij elke PR (b.v. Snyk / CodeQL)
- SCA: automatische dependency alerts (b.v. Snyk / Dependabot)
- SBOM: Software Bill Of Materials als artefact bij build (b.v. CycloneDX formaat)
- OTAP-omgevingen: Ontwikkeling, Test, Acceptatie, Productie gescheiden

Deze workshop: Niet inrichten (dat is al gedaan) maar **verantwoorden**.

Terugblik WS 2 | Guardrails

Pipeline-artefact	Sectie in auditrapport
Branch protection ruleset (screenshot)	Bevinding: "Broncode is beveiligd"
CodeQL SARIF-output	SAST-bijlage + bevindingen
Dependabot alerts log	SCA-bewijs + kwetsbaarheidsbeheer
SBOM (CycloneDX JSON)	SBOM & Supply Chain Security sectie
PR review history	Traceability matrix (na-bewijs)

Terugblik WS 3 | Risk Assessment

Geinformeerde keuzes maken

- Hack-driehoek: $\text{Motivatie} \times \text{Middelen} \times \text{Kans} = \text{bedreiging}$
- Risicomatrix: $\text{Kans} \times \text{Impact} \rightarrow \text{risiconiveau}$
 - Kwalitatief (laag/midden/hoog/kritiek) / Kwantitatief (1 / 5 / 7 / ...)
- ISO 27005 als methode voor risicoanalyse
- Bow-tie diagram
 - oorzaken $\rightarrow$ incident $\rightarrow$ gevolgen + NEN-7510 controls als barrières

Terugblik WS 4 | Compliance scanning

‘Bewijs verzamelen’

- CVE/CVSS
 - Common Vulnerabilities and Exposures
 - Common Vulnerability Scoring System
- Snyk workflow
 - scannen van dependencies op bekende kwetsbaarheden
- Security backlog
 - geprioriteerde lijst van bevindingen
- Risk assessment rapport
 - structuur en inhoud

De risicomatrix + de Snyk CVE-lijst + de bow-tie → kern van sectie 4 (Risico-analyse & Bevindingen) van het auditrapport.

'Bevindingen mitigeren'

- OWASP Top 10
 - de meest voorkomende kwetsbaarheden in webapplicaties
- Attack surface analysis
 - welke entry points heeft de applicatie?
- Privacy by Design
 - privacy als architectuurprincipe, niet als bijgedachte
- Logging in code
 - correct en compliant logging implementeren (de 5 W's)
- Unit tests voor security
 - aantonen dat een kwetsbaarheid gefixt is

Voor (bevinding)

SQL Injection
in PatientSearch.java

Na (bewijs)

PR #42: parameterized query
+ herhaalde SAST scan:
 geen bevindingen
+ unit test:
 SqlInjectionTest.java

Tracebility matrix

Fix → 'Na' kolom

SAST result → 'Voor' kolom

Traceability | bewijs $\leftarrow \rightarrow$ claim

"Een rapportage zonder bewijs is maar een mening."

- audit-rapportage: elke **claim** moet worden ondersteund door een **artefact**.

Aantoonbare verbinding tussen:

- Een **norm of eis** (bijv. NEN-7510 8.8)
- Een **maatregel** (bijv. Dependabot ingeschakeld)
- Een **bewijs** (bijv. Dependabot alerts log van 2024-02-01)

Traceability | bewijs $\leftarrow \rightarrow$ claim

Claim	Bewijs
"We beheren technische kwetsbaarheden"	Dependabot-alerts log + SBOM + patch history (commit hash)
"Broncode is beveiligd tegen ongeautoriseerde wijzigingen"	Branch protection ruleset screenshot + PR review history
"Logging voldoet aan NEN-7510 8.15"	Code snippet + unit test output + log sample (geanonimiseerd)
"Er zijn geen SQL-injecties in de module"	CodeQL SARIF-output na fix + PR #42 + SqlInjectionTest.java
"Supply chain risico's zijn geïntinventariseerd"	CycloneDX SBOM JSON + Snyk rapport met datum

NEN-7510 5.35: Onafhankelijke beoordeling vereist gedocumenteerd bewijs.

- 'We hebben dit gedaan' is geen bewijs

Traceability gotcha's

✗ Claim zonder bewijs

"Onze applicatie is veilig tegen SQL-injectie." → Waar is het bewijs? Een SAST-scan? Een pentest? Een code review?

✗ Bewijs zonder claim

[Bijlage: 200 pagina's Snyk output zonder duiding] → Wat bewijst dit? Welke control wordt hiermee aangetoond?

✗ Verouderd bewijs

"We hebben in 2022 een security scan gedraaid." → Software verandert. Bewijs moet actueel zijn.

✗ Bewijs dat de claim niet dekt

"Toegang tot medische dossiers is gelogd" → Bewijs: server-uptime rapport. Dit dekt de claim niet.

✓ **Claim:** "Kwetsbare dependencies worden binnen 30 dagen gepatcht."

Bewijs: Dependabot alert CVE-2021-44228 geopend op 2024-01-10, PR #38 gemerged op 2024-01-15 (5 dagen). Zie bijlage C.

PoC | bevindingen ↔ mitigaties

- Proof of Concept (security context) is een minimale, reproduceerbare demonstratie dat een kwetsbaarheid daadwerkelijk uitbuitbaar is én dat een mitigatie die uitbuiting effectief verhindert.

Wel	Niet
Bewijs dat de kwetsbaarheid exploiteerbaar is	Een volledig werkende exploit voor productie
Minimale reproduceerbare stappen	Een uitgebreide aanvalstool
Verificatie dat de fix werkt (hertest)	Een theoretische beschrijving zonder bewijs
Gedocumenteerd, herhaalbaar	Eenmalig handmatig getest

Doel

- Aantonen dat de kwetsbaarheid gevonden *en* opgelost is
- Bewijs leveren voor de auditor (NEN-7510 8.29)

PoC | Stappenplan

1. Documenteer de kwetsbaarheid
 - Beschrijf de kwetsbaarheid (type, locatie in code, CVSS)
 - Voer de eerste pentest uit
 - Leg de uitkomst vast: HTTP-response, screenshot, ZAP-alert
 - Bewijs: "Dit is de kwetsbaarheid, dit is de impact"
2. Implementeer de mitigatie
 - Schrijf de fix (code-wijziging, configuratiewijziging, update)
 - Code review door tweede persoon
 - Unit test of integratietest toevoegen
 - Commit met referentie naar bevinding #nummer
3. Voer de hertest uit
 - Herhaal exact dezelfde teststappen als stap 1
 - Gebruik dezelfde tool (ZAP), dezelfde parameters, dezelfde gebruiker
 - Leg de nieuwe uitkomst vast: HTTP-response, screenshot, ZAP-alert
 - Bewijs: "De aanval geeft nu [403 / 400 / foutmelding] i.p.v. [200 / exploiteerbaar]"
4. Beschrijf de delte
 - Wat was het gedrag vóór de fix?
 - Wat is het gedrag ná de fix?
 - Zijn er neveneffecten of restrisico's?
 - Update de status van de bevinding naar "Geverifieerd opgelost"

Verantwoording AI tooling

Waarom?

- Aantonen dat **jij** de security-beslissingen hebt genomen, niet de AI.
 - AI-tools zijn hulpmiddelen
 - jij bent de professional die verantwoording draagt.

Format verantwoording AI tooling

AI-TOOLING VERANTWOORDING

Gebruikte tools: GitHub Copilot, ChatGPT (GPT-4o)

Wat ik aan AI heb gevraagd:

- "Genereer een Java-methode die autorisatie controleert voor FHIR Patient resources in OpenMRS"
- "Leg uit wat BOLA is en geef een voorbeeld"

Wat de AI heeft gegenereerd:

- Conceptcode voor de autorisatiecheck (zie bijlage A)
- Uitleg van BOLA met een generiek HTTP-voorbeeld

Wat ik zelf heb gecontroleerd:

- Of de AI-code de juiste OpenMRS API-methoden aanroept (Context.getAuthenticatedUser() – bevestigt via broncode)
- Of de fix geen regressie introduceert (integratietest)
- Of de CVSS-score correct is berekend (handmatig nagelopen)

Beslissingen die ik zelf heb gemaakt:

- Keuze voor 403 (niet 404) als HTTP-foutcode
- Scope van de fix (alleen FHIR2, niet de REST-API module)
- Beoordeling dat de residuele risico's acceptabel zijn

Voorbeeld format PoC

Proof of Concept – Bevinding #[N]: [Titel]

1 Initiële Bevinding

- Datum test: [datum]
- Tester: [naam]
- Tool: OWASP ZAP 2.14 + handmatige verificatie
- Kwetsbaarheid: BOLA/IDOR – OWASP A01:2021
- CVSS v3.1 Score: 7.5 (High)
 - Vector: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
- Bewijs: [Screenshot 1 – HTTP 200 response met patiëntdata]

2 Geïmplementeerde Mitigatie

- Type: Code-wijziging in `FhirPatientResourceProvider.java`
- Commit: `git commit abc1234 – fix: voeg autorisatiecheck toe aan FHIR Patient GET`
- Beschrijving: Autorisatiecheck toegevoegd die valideert of de ingelogde gebruiker toegang heeft tot de gevraagde patiënt via `authorizationService.canViewPatient()`

3 Hertest

- Datum hertest: [datum]
- Tool: OWASP ZAP 2.14 + handmatige verificatie (identieke teststappen)
- Resultaat: HTTP 403 Forbidden (voorheen: HTTP 200 OK)
- Bewijs: [Screenshot 2 – HTTP 403 response]

4 Delta en Restrisico

- Resultaat: Kwetsbaarheid opgelost
- Neveneffecten: Geen — regressietests geslaagd
- Restrisico: Autorisatielogica in REST-module (webservices.rest) nog niet getest → Aanbeveling: vergelijkbare check uitvoeren op `/ws/rest/v1/patient/{uuid}`
- NEN-7510 8.29: Verificatie van beveiligingscontroles gedocumenteerd en aantoonbaar

Doelgroepen (aka 'read the room')

MANAGEMENT (Raad van Bestuur, Bestuurders, Compliance officers)	TECHNISCH TEAM (Developers, Ops, Security engineers)
Focus: • Risico's voor de org. • Reputatieschade • Boetes (AVG, CRA) • Patiëntveiligheid • "In control zijn"	Focus: • CVE-ID's en CVSS-scores • Specifieke code-regels • Reproduceerbare stappen • Patch-instructies • Tool-configuraties
Taal: Zakelijk, helder, resultaatgericht	Taal: Technisch, actiegericht, reproduceerbaar
Format: Executive summary RAG-status Top 3 risico's Roadmap (hoog over)	Format: Gedetailleerde bevindingen Traceability matrix Tool output als bijlage Commit hashes, PR-nummers

RAG: Rood / Amber / Groen

Rapportage richting management

executive summary: (max) één pagina, maximale impact

Wat wil management weten?

- Zijn we "in control"? (RAG-status: Rood / Amber (Oranje) / Groen)
- Wat zijn de grootste risico's voor onze organisatie?
- Wat kost het als we niets doen? (boetes, reputatie, continuïteit)
- Wat moet er gebeuren, en wanneer?

RAG-status:

 **Rood:** Kritieke bevindingen aanwezig, patiëntdata of continuïteit in gevaar

 **Oranje:** Significante bevindingen, risico beheerst maar verbetering nodig

 **Groen:** Bevindingen aanwezig maar laag risico, in control

Rapportage richting management

Niet	Wel
"Er is een SQL-injectie kwetsbaarheid in PatientSearch.java"	"Patiëntgegevens kunnen worden uitgelezen door onbevoegden via de zoekfunctie"
"Log4j CVE-2021-44228 CVSS 10.0 aanwezig in dependency tree"	"Een kritieke beveiligingsfout in een gebruikte softwarecomponent maakt volledige serverovername mogelijk"
"Branch protection rules zijn niet geconfigureerd"	"Medewerkers kunnen ongecontroleerd aanpassingen doen aan de productie-software"
"SBOM ontbreekt"	"Er is geen overzicht van alle gebruikte softwarecomponenten, waardoor kwetsbaarheden onopgemerkt kunnen blijven"

- ***Note:** management is NIET dom maar druk. Een CTO of Raad van Bestuur heeft beperkte tijd voor dit rapport. Maak optimaal gebruik van die tijd!
- ****Note:** **note is onder dwang toegevoegd*

Rapportage richting technisch team



Gedetailleerde bevindingen: reproduceerbaar en actiegericht

Bevinding #4 – SQL Injection in PatientSearchController.java

Ernst: Kritiek (CVSS 3.1: 9.8 – AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
NEN-7510: 8.28 Veilig coderen
Gevonden door: CodeQL SAST, scan 2024-02-01
Status: Opgelost (PR #42, 2024-02-08)

Beschrijving:

De methode `getPatientsByName()` in `PatientSearchController.java` (regel 87) construeert een SQL-query door directe string-concatenatie van user input. Dit maakt het mogelijk voor een aanvaller om arbitraire SQL te injecteren.

Kwetsbare code (vóór):

```
String query = "SELECT * FROM patient WHERE name = '" + userInput + "'";
```

Gecorrigeerde code (na):

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM patient WHERE name = ?");
stmt.setString(1, userInput);
```

Bewijs van oplossing:

- PR #42: <https://github.com/org/openmrs-module/pull/42>
- Herhaalde CodeQL scan: bijlage D (nul SQL injection findings)
- Unit test: `SqlInjectionTest.java#testMaliciousInput()` – PASSED

Aanbeveling:

Controleer alle andere DAO-klassen op vergelijkbaar patroon.
Overweeg een linting-regel die directe string-concatenatie in SQL verbiedt.

Developer die dit leest weet: 1 wat het probleem is, 2 waar het zit, 3 hoe het opgelost is, 4 hoe te verifiëren.
Reproduceerbaar = actionable.

Structuur auditrapport

7 secties

1. Executive Summary (max 1 pagina) RAG-status, top risico's, roadmap
2. Scope & Context Wat is getest, welke versie, welk normenkader
3. Audit Methodologie Welke tools, welke periode, welke aanpak
4. Risico-analyse & Bevindingen Per bevinding: ID, CVSS, control, bewijs
5. SBOM & Supply Chain Security Top dependencies, bekende CVE's
6. Conclusie & Advies Nu / deze sprint / later
7. Bijlagen Tool output, screenshots, SBOM JSON

Gebruik dit als referentie!

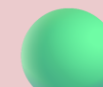
1 | Executive Summary

(max) één pagina, maximale impact

Wat wil management weten?

- Overall status. Zijn we "in control"?
- Top risico's voor continuïteit
- Geprioriteerde roadmap
- Wat moet er gebeuren, en wanneer?

#	Risico	Ernst	Status
1	SQL-injectie mogelijke toegang tot patiëntdossiers	 Kritiek	Opgelost (PR #42)
2	Verouderde Log4j-component (CVE-2021-44228)	 Kritiek	Opgelost (Dependabot PR #38)
3	Ontbrekende audit trail bij inzage patiëntdossier	 Hoog	In behandeling

Prioriteit	Actie	Termijn
 Nu	Valideer alle overige DAO-klassen op SQL-patronen	Direct
 Deze sprint	Implementeer audit logging voor dossierinzage	Sprint 3
 Later	Voer DAST-scan uit op acceptatie-omgeving	Q2 2024

Wordt als laatste geschreven maar staat als eerste in rapport.

2 | Scope & context

Beschrijf WAT beoordeeld is:

- Module/applicatie:
 - naam, versie, repository-URL
- Beoordelingsperiode:
 - van ... tot ...
- Testomgeving:
 - waar in OTAP (Development / Test / Acceptatie)
- Wat is buiten scope:
 - b.v. productie-infrastructuur, andere modules
- Norm: NEN-7510:2024 Deel 2
 - welke controls zijn van toepassing?
- Wetgeving:
 - AVG, NIS2/Cyberbeveiligingswet, CRA (indien van toepassing)

Neem OOK op wat NIET getest is, en waarom.

3 | Audit methode

Beschrijf HOE getest is

Techniek	Tool	Wanneer	Resultaat
SAST	CodeQL (GitHub Actions)	Bij elke PR + eindmeting	SARIF-rapport, zie bijlage A
SCA	Dependabot + Snyk	Wekelijks + eindmeting	Alerts log, zie bijlage B
SBOM	CycloneDX Maven Plugin	Build-tijd	sbom.json, zie bijlage C
Code review	Handmatig (peer review)	WS05	PR-commentaar, zie bijlage D
Risicoanalyse	ISO 27005 methode	WS03	Risicomatrix, zie bijlage E

4 | Bevindingen

Elke bevinding volgens 'vast' formaat

Ook: Contextuele score: impact kan hoger / lager zijn dan CVSS aangeeft.

b.v.

Bevinding ID:	B-004
Titel:	Ontbrekende audit trail bij dossierinzage
Datum gevonden:	2024-02-03
Status:	Open
CVSS-score:	6.5 (Medium)
CVSS-vector:	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Contextuele score:	7.5 (Hoog) – hogere impact in zorgcontext omdat het AVG-verantwoording blokkeert
NEN-7510 Control:	8.15 Logregistratie
Beschrijving:	Bij het opvragen van een patiëntdossier via de PatientRecordService wordt geen log-event gegenereerd. Hierdoor kan niet worden gereconstrueerd wie welk dossier heeft ingezien (AVG art. 30 verantwoordingsplicht).
Bewijs:	Code review PatientRecordService.java r.112 (geen auditLogger.log() aanroep aanwezig) Screenshot: bijlage F
Aanbeveling:	Implementeer audit logging bij iedere opvraag: timestamp, gebruiker-ID, patiënt-ID, actie, bron-IP. Zie WS05 logging-richtlijn.

5 | SBOM & Supply Chain security

Het fundament van:

- **Kwetsbaarheidsbeheer:** weet welke versie er draait
- **CRA-compliance:** SBOM beschikbaar stellen is een CRA-verplichting
- **Leveranciersbeheer:** NEN-7510 5.22 Monitoring leveranciers
- **NEN-7510 8.8:** Beheer van technische kwetsbaarheden:

Component	Versie	Licentie	Bekende CVE's	Status
log4j-core	2.14.1	Apache-2.0	CVE-2021-44228 (CVSS 10.0)	✓ Gepatcht → 2.17.1
spring-webmvc	5.3.20	Apache-2.0	Geen actief	✓ OK
mysql-connector-java	8.0.28	GPL-2.0	CVE-2022-21363 (CVSS 5.9)	⚠ Update aanbevolen
jackson-databind	2.13.1	Apache-2.0	CVE-2022-42003 (CVSS 7.5)	● Open — zie B-007

6 | Conclusie & advies

antwoord op de auditervraag:

- *Voldoet de OpenMRS-module aan de relevante NEN-7510:2024-2 controls en CRA-verplichtingen?*
- **Prioritering van aanbevelingen:**

Prioriteit	Criterium	Actie
 Nu	Kritieke of hoge bevinding, nog open	Direct oplossen voor productie-deployment
 Deze sprint	Gemiddelde bevinding, plan aanwezig	Inplannen in aankomende sprint
 Later	Lage bevinding of lange-termijn verbetering	Opnemen in security roadmap

7 | Bijlagen

Elke bijlage heeft een ID en een beschrijving, b.v.:

- Bijlage A: CodeQL SARIF-output, scan 2026-02-01 (eindmeting)
- Bijlage B: Dependabot alerts log, periode jan–feb 2026
- Bijlage C: CycloneDX SBOM JSON (sbom.json), build 2026-02-08
- Bijlage D: PR-overzicht met review-commentaar (WS05)
- Bijlage E: Risicomatrix (WS03) en bow-tie diagram
- Bijlage F: Screenshots code review bevindingen
- Bijlage G: Snyk-rapport, scan 2026-02-01

Bijlagen zijn niet decoratief maar zijn het bewijs.

- Elke bijlage moet worden gerefereerd vanuit de hoofdtekst.
- Een bijlage zonder referentie heeft geen auditwaarde.

*CRA
VS
NEN-7510*

avans
hogeschool

CRA vs NEN-7510

De Cyber Resilience Act (CRA) (in het kort)

- De CRA is een EU-verordening (van kracht 2024, volledig van toepassing 2027) die eisen stelt aan producten met digitale elementen — software, hardware en embedded systemen die verbonden zijn of verbinding kunnen maken.
- OpenMRS-modules zijn "producten met digitale elementen" → CRA is van toepassing.
- Vraag: Als NEN-7510 toepast is, is dan ook de CRA afgedekt?

CRA vs NEN-7510

CRA Verplichting	NEN-7510:2024-2 Control
Software leveren zonder bekende (actieve) kwetsbaarheden	8.8 Beheer van technische kwetsbaarheden
SBOM beschikbaar stellen aan gebruikers	8.8 + 5.22 Monitoring leveranciers
Beveiligingsupdates leveren gedurende de levensduur	8.8 Patch management
Secure by design — veiligheid als standaard	8.25 Beveiligen tijdens de ontwikkelcyclus
Actief misbruikte kwetsbaarheden melden aan ENISA	6.8 Rapportage van informatiebeveiligingsgebeurtenissen
Logging en monitoring ondersteunen	8.15 Logregistratie + 8.16 Monitoringactiviteiten
Toegangscontrole voor beheerinterfaces	8.2 Beheer van bevoorrechte toegangsrechten

Meldplicht (ENISA)?

Responsible Disclosure

Je vindt een kritieke kwetsbaarheid. En nu?

- Bij het testen van de OpenMRS-module ontdekt dat CVE-2023-99999 (CVSS 9.8) aanwezig is in een library die in productie draait bij drie ziekenhuizen. Er is nog geen patch beschikbaar.

Dilemma:

- Onmiddellijk publiceren: aanvallers worden gewaarschuwd, maar ook geholpen
- Nooit publiceren: de kwetsbaarheid blijft aanwezig, patiënten blijven risico lopen
- Verantwoord bekendmaken: gestructureerd proces, vendor krijgt kans om te patchen

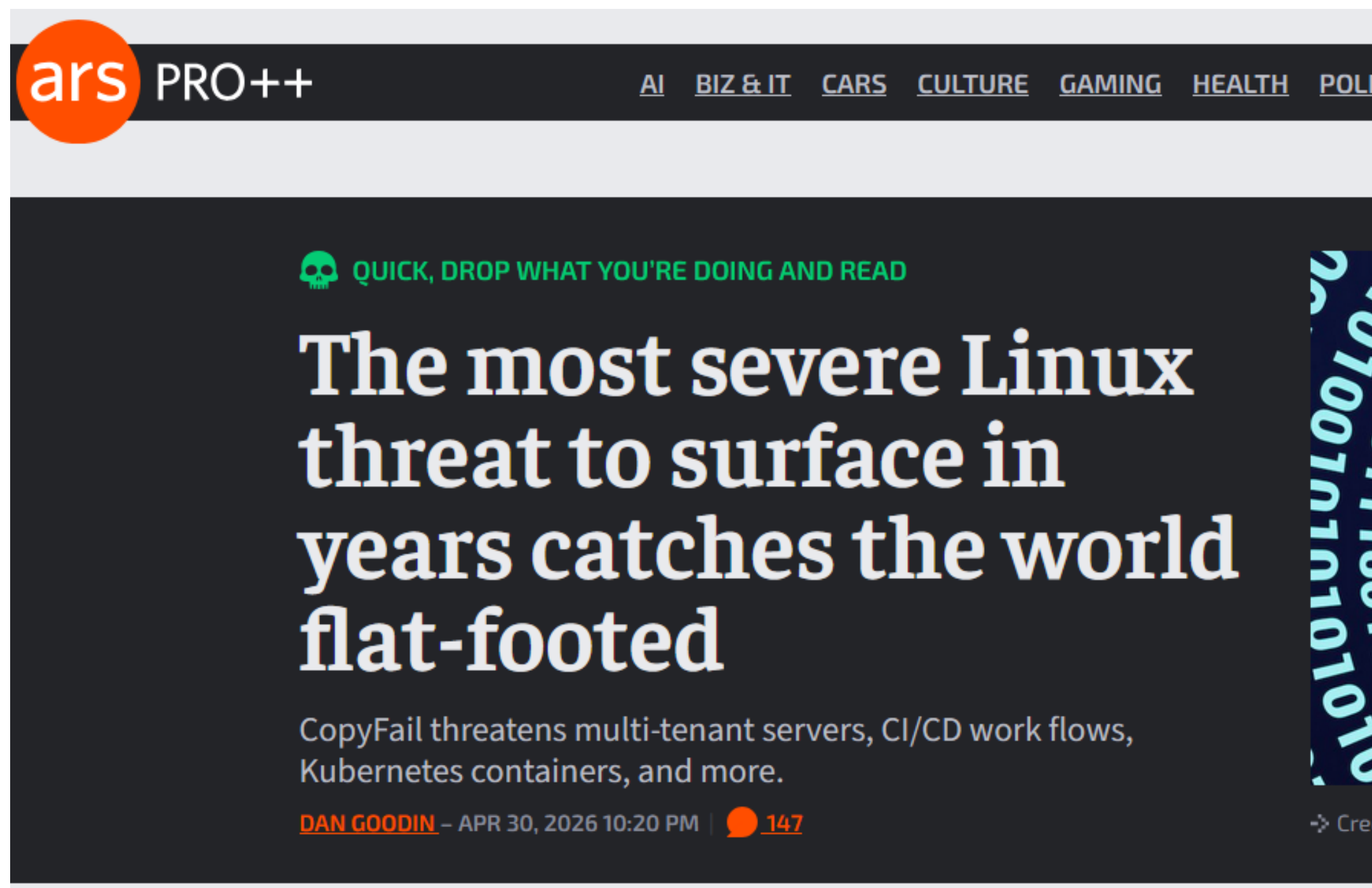
Definitie responsible disclosure:

- Het verantwoord bekendmaken van kwetsbaarheden aan de betrokken partijen (vendor, CERT, gebruikers) op een manier die schade minimaliseert en de kans op een fix maximaliseert, vóórdat publieke openbaarmaking plaatsvindt.

Responsible Disclosure

30 april

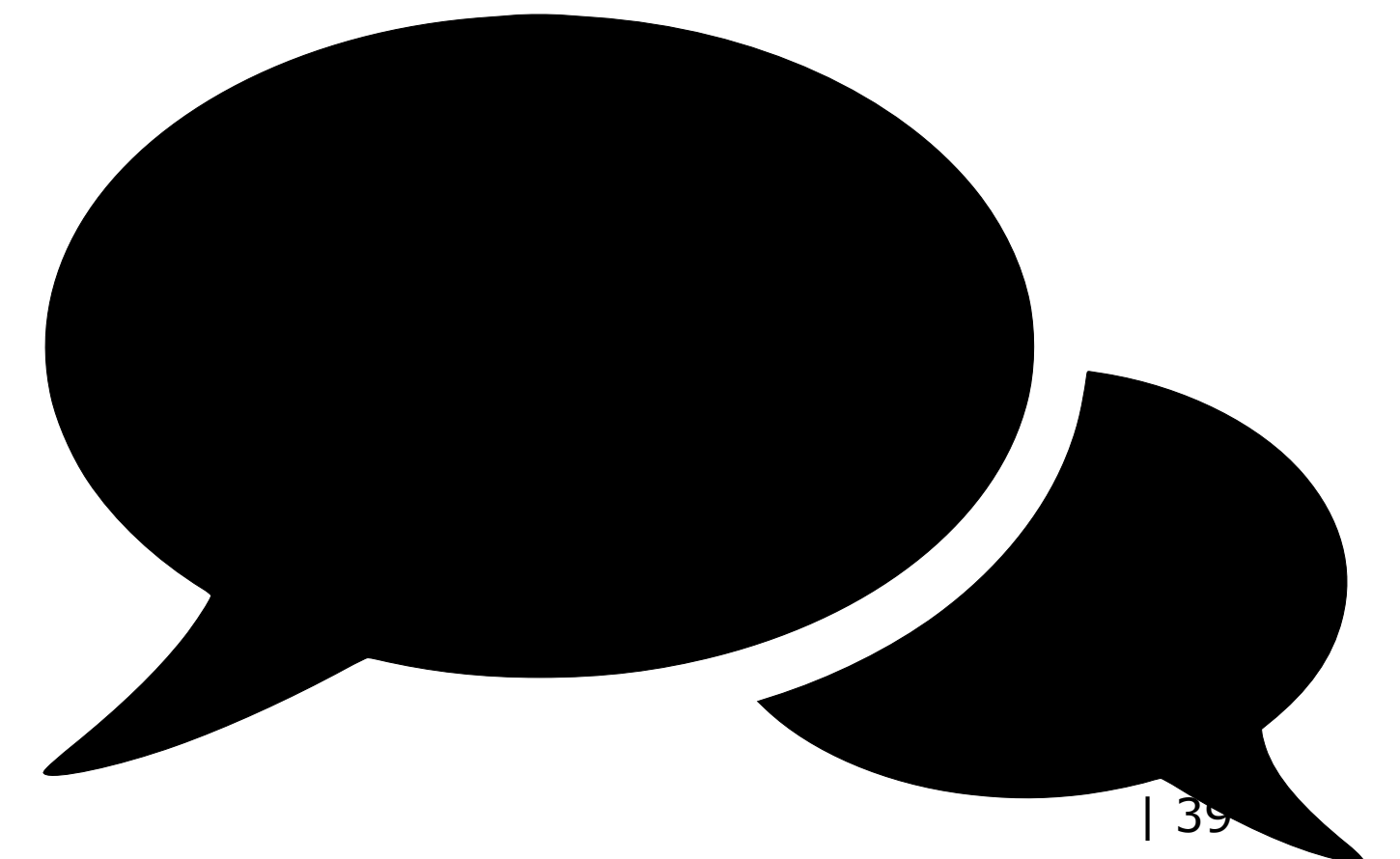
1 mei...Ohohhh



Responsible Disclosure

Wettelijk kader:

- **NEN-7510 6.8:** Rapportage van informatiebeveiligingsgebeurtenissen — incidenten en kwetsbaarheden moeten worden gerapporteerd
- **CRA Art. 14:** Verplichte melding van actief misbruikte kwetsbaarheden aan ENISA binnen 24 uur
- **Wet computercriminaliteit III:** Responsible disclosure schept geen volledige immuniteit, maar toont goede trouw



Responsible Disclosure | stappen

STAP 1: Documenteer

Wat heb je gevonden? Reproduceerbare beschrijving.
CVSS-score berekenen. Welke systemen zijn getroffen?



STAP 2: Neem contact op met de vendor

Zoek het security-contactadres (security@vendor.com of security.txt op de website). Stuur een versleuteld bericht (PGP indien beschikbaar). Geef de vendor 90 dagen.



STAP 3: Geef een redelijke hersteltermijn

Standaard: 90 dagen (Google Project Zero)
Bij actief misbruik: kortere termijn (bijv. 7-14 dagen)
Bij kritieke infrastructuur: coördineer met NCSC



STAP 4: Coördineer via CERT/CC of NCSC

Het Nationaal Cyber Security Centrum (NCSC) kan bemiddelen als de vendor niet reageert of de urgentie onvoldoende erkent.
CERT Coordination Center (CERT/CC) voor internationale cases.



STAP 5: Publiceer na patch of na termijn

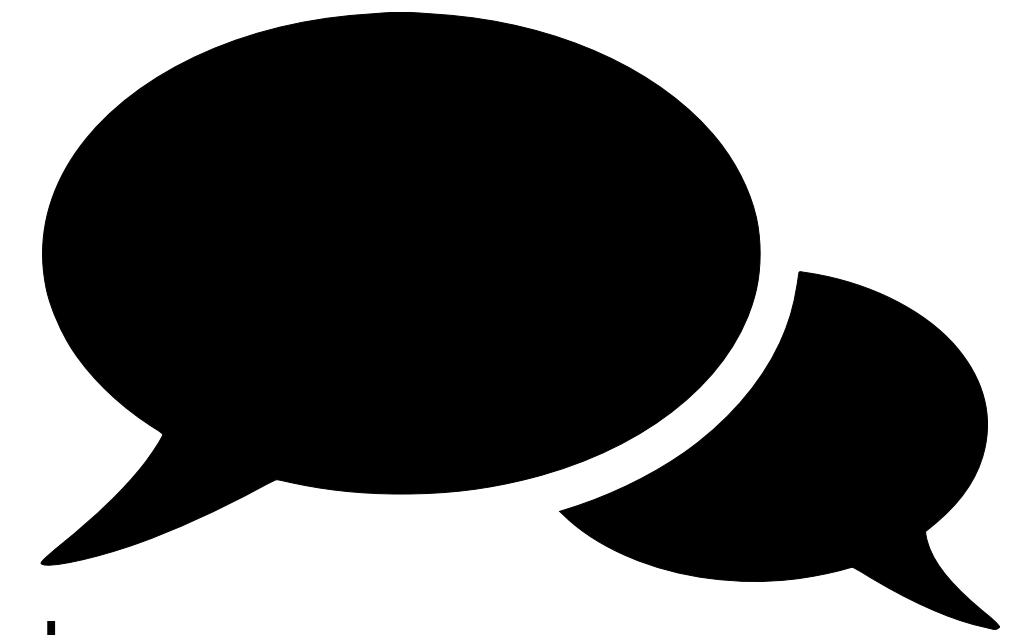
Als de vendor een patch heeft uitgebracht: publiceer CVE + fix.
Als de termijn verstreken is zonder patch: publiceer met vermelding van de omstandigheden.

- **CVSS:** Kritieke kwetsbaarheden (CVSS 9.0+) rechtvaardigen een kortere hersteltermijn.

Bij actief misbruik is urgentie maximaal.

Scenario: je bent de ontdekker

- **Situatie:** Tijdens je stage bij een zorginstelling analyseer je de OpenMRS-installatie. Je ontdekt dat de gebruikte versie van commons-text kwetsbaar is voor CVE-2022-42889 ("Text4Shell", CVSS 9.8). Deze kwetsbaarheid maakt RCE mogelijk. De module draait op de acceptatie-omgeving, maar de systeembeheerder heeft bevestigd dat een identieke versie op productie staat bij twee aangesloten klinieken.
- Er is een patch beschikbaar (commons-text 1.10.0).
- **Vraag:** Wat doe je, in welke volgorde, en aan wie rapporteer je?
- **Vraag:** NEN-7510 Relevantie?
 - 6.8: Dit is een informatiebeveiligingsgebeurtenis die formeel gerapporteerd moet worden.



Traceability Matrix



Koppelt elke norm aan control en bewijs (voor en na)

Norm	Maatregel	Vóór (bevinding)	Aanpassing	Na (bewijs)
NEN-7510 8.28 Veilig coderen	Inputvalidatie via parameterized queries	SQL injection in PatientSearch.java regel 87 (CodeQL, scan 2024-02-01)	Parameterized query geïmplementeerd — PR #42, 2024-02-08	Herhaalde CodeQL scan 2024-02-09: geen SQL injection findings. Unit test SqlInjectionTest.java PASSED
NEN-7510 8.8 Kwetsbaarheidbeheer	Dependabot ingeschakeld, patch binnen 30 dagen	Log4j 2.14.1 (CVE-2021-44228, CVSS 10.0) aanwezig in dependency tree (Snyk, 2024-01-10)	Dependency geüpdatet naar log4j-core 2.17.1 — PR #38, 2024-01-15	SBOM bijgewerkt, Snyk scan 2024-01-16: CVE-2021-44228 niet meer aanwezig
NEN-7510 8.15 Logregistratie	Audit logging bij dossierinzage	Geen log-events bij PatientRecordService.getRecord() (code review, 2024-02-03)	AuditLogger.logAccess() aanroep toegevoegd — PR #47, 2024-02-10	Log sample bijlage F toont timestamp, user-ID, patiënt-ID, actie. Unit test AuditLogTest.java PASSED
NEN-7510 8.25 Ontwikkelcyclus beveiliging	Branch protection + verplichte PR review	Directe push naar main was mogelijk (GitHub audit log, 2024-01-08)	Branch protection rule ingesteld: 2 reviewers vereist, CodeQL check vereist — WS02	GitHub branch protection screenshot bijlage G. Geen directe pushes sinds 2024-01-09
NEN-7510 5.35 Onafhankelijke beoordeling	Dit auditrapport zelf	Geen formele beoordeling aanwezig vóór WS06	Auditrapport opgesteld conform NEN-7510:2024-2 scope	Dit document, inclusief alle bijlagen

Traceability Matrix

Koppelt elke norm aan control en bewijs (voor en na)

Kolom	Doel	Valkuil
Norm	Welke NEN-7510 control (of CRA-eis) wordt aangetoond?	Te vaag: "beveiliging" in plaats van "8.28 Veilig coderen"
Maatregel	Welke concrete maatregel is genomen?	Te generiek: "we zijn veiliger geworden"
Vóór (bevinding)	Wat was de situatie vóór de maatregel? Wat was het probleem?	Geen datum, geen tool-referentie, geen bestandsnaam
Aanpassing	Wat is er concreet gewijzigd?	PR-nummer ontbreekt, geen commit hash
Na (bewijs)	Hoe toon je aan dat de maatregel werkt?	"We geloven dat het opgelost is" — bewijs het!

Gebruik de bevindingen (b.v. Snyk-rapport) als "Vóór"

Gebruik de code-fixes als "Aanpassing"

Gebruik de herhaalde scans als "Na"

Elk item in de "Na"-kolom moet een verifieerbaar artefact zijn: een commit hash, PR-nummer, scan-output met datum, test-resultaat, screenshot, ...

Vertalen | Technisch naar Audit-taal

Zelfde feiten, verschillend verhaal

Technische Output	Audit Bevinding	Risico voor de Zorg
SQL Injection in PatientSearch.java (CodeQL, regel 87)	Onvoldoende inputvalidatie — gebruikersinvoer wordt ongevalideerd verwerkt in databasequery's (NEN-7510 8.28)	Onbevoegde toegang tot volledige patiëntendatabase; massale datalekken mogelijk; AVG boetes tot €20M of 4% jaaromzet
log4j-core:2.14.1 — CVE-2021-44228, CVSS 10.0, Remote Code Execution	Gebruik van componenten met bekende kritieke kwetsbaarheden in de leveringsketen van de applicatie (NEN-7510 8.8)	Volledige serverovername door aanvallers; ransomware-risico voor ziekenhuissystemen; zorgcontinuïteit in gevaar
Hardcoded API key in config.properties regel 23	Onveilig beheer van cryptografische sleutels en wachtwoorden — credentials opgeslagen in broncode (NEN-7510 8.28)	Compromittering van externe koppelingen met apotheeksystemen; onbevoegde toegang tot medicatie-informatie
Geen auditLogger.log() aanroep in PatientRecordService.getRecord()	Ontbrekende logregistratie voor toegang tot medische dossiers — inzage wordt niet bijgehouden (NEN-7510 8.15)	Onmogelijk te reconstrueren wie welk dossier heeft ingezien; AVG verantwoordingsplicht (art. 30) kan niet worden nagekomen; forensisch onderzoek bij incident onmogelijk

Eindproduct | Auditrapport

Drie deliverables

1. Executive Summary

- Doelgroep: Directie / besluitvormers
- Inhoud: RAG-status, top 3 risico's, roadmap

2. Traceability Matrix

- Minimaal 5 NEN-7510:2024-2 controls
- Voor / aanpassing / na (met concreet bewijs)

3. Volledig audit rapport

- Alle 7 secties + bijlagen
- Koppeling aan CRA waar van toepassing

Oplevering: Markdown of Word-document, opgeslagen als artefact in de repository (niet als e-mail bijlage — het rapport is onderdeel van je software-levering).

Deliverable 1 | Executive summary

Instructies:

- Schrijf voor een Raad van Bestuur die geen technische achtergrond heeft
- Gebruik geen jargon (geen CVE-nummers, geen CVSS in de tekst — die horen in het technische rapport)
- Begin met de overall RAG-status en een éénzin-samenvatting
- Benoem de top 3 risico's in begrijpelijke taal
- Sluit af met een geprioriteerde roadmap (3 categorieën)
- Max 400 woorden

Criteria:

- Zijn de risico's vertaald naar zorg-impact (niet naar technische impact)?
- Is de RAG-status onderbouwd?
- Is de taal geschikt voor niet-technische besluitvormers?
- Zijn de aanbevelingen concreet en actionable?

Deliverable 1 | Executive summary

Checklist:

- [] Overall RAG-status (● / ● / ●) met korte toelichting
- [] Status t.o.v. CRA en NEN-7510:2024-2
- [] Top 3 risico's voor continuïteit (niet voor de software)
- [] Geprioriteerde roadmap: nu / deze sprint / later
- [] Maximaal 400 woorden

Deliverable 2 | Traceability Matrix

- Minimaal 5 NEN-7510:2024-2 controls
- Per rij: norm, maatregel, vóór (bevinding), aanpassing, na (bewijs)
- Elk bewijs moet een concreet artefact zijn (PR-nummer, scan-datum, commit hash)
- Aanbevolen controls:
 - 8.28 Veilig coderen (SAST-bevindingen)
 - 8.8 Kwetsbaarheidbeheer (Dependabot/Snyk-bevindingen)
 - 8.15 Logregistratie (logging-implementatie WS05)
 - 8.25 Beveiliging in de ontwikkelcyclus (pipeline WS02)
 - 5.35 of 5.36 Naleving/beoordeling (dit rapport zelf)

Deliverable 3 | Audit rapport

Alle 7 secties uit de rapportstructuur (zie eerder)

- Management Samenvatting (= Deliverable 1, geïntegreerd)
- Scope & Context
- Audit Methodologie
- Risico-analyse & Bevindingen (minimaal 4 bevindingen)
- SBOM & Supply Chain Security
- Conclusie & Advies
- Bijlagen:
 - SBOM (b.v. CycloneDX JSON)
 - SAST-output (b.v. CodeQL of Snyc)
 - Risicomatrix
 - Bow-tie diagrammen / threat models
 - Snyc-rapport
 - CRA-mapping

FAQ



- V: Moet de module al helemaal veilig zijn voor ik het rapport schrijf?
- A: Nee. Een auditrapport documenteert de situatie inclusief openstaande bevindingen. Open bevindingen zijn geen fout; ze tonen aan dat ze gevonden zijn en er een plan is. Wat je NIET mag: bevindingen verzwijgen.

- V: Bevindingen opnemen die ik nog niet heb opgelost?
- A: Ja, sterker nog: je moet ze opnemen. Een bevinding heeft status "Open". Geef de prioritering aan.

- V: Hoe lang moet het rapport zijn?
- A: Kwaliteit boven kwantiteit. Een beknopt, goed onderbouwd rapport van 15 pagina's + bijlagen is beter dan een 40-pagina's rapport vol filler. De executive summary is maximaal 400 woorden.

- V: AI-tools voor het schrijven?
- A: Ja, als hulpmiddel. De bevindingen, het bewijs en de analyse moeten van jou zijn. AI-gegenereerde tekst zonder eigen analyse is niet acceptabel.

- V: Hoe specifiek moet de SBOM-sectie zijn?
- A: Benoem minimaal de top 10 dependencies op risico (hoogste CVSS-score), plus alle bekende CVE's die aanwezig waren. Voeg de volledige SBOM toe als bijlage.

- V: Welk normenkader gebruik ik als primaire referentie?
- A: NEN-7510:2024 Deel 2 is het primaire kader. CRA is aanvullend, specifiek voor de leveranciersverplichting. AVG is relevant voor privacy/logging-controls.

Compliance framing

Alle 7 secties uit de rapportstructuur (zie eerder)

- Management Samenvatting (= Deliverable 1, geïntegreerd)

Compliance als afvinklijstje	Compliant by design
"We hebben CodeQL ingesteld, klaar."	"CodeQL helpt ons continue kwaliteit garanderen."
"We hebben een SBOM gemaakt voor de audit."	"De SBOM is onderdeel van elke build."
"We loggen omdat de norm het vereist."	"We loggen omdat we incidenten willen kunnen reconstrueren."
"We doen een risicoanalyse eens per jaar."	"Risico's zijn onderdeel van elke sprint planning."

Change of View (hopelijk?)

Is dit waar?

- **Je kijkt anders naar dependencies.**
 - Elke pom.xml of package.json is een risico-inventarisatie. Je denkt aan de SBOM, aan bekende CVE's, aan het update-beleid van de vendor.
- **Je schrijft code met de auditor in gedachten.**
 - Inputvalidatie, parameterized queries, foutafhandeling zonder stack traces in de response.
- **Je begrijpt waarom logging bestaat**
 - Niet als debugging-hulpmiddel, maar als forensisch spoor. Wie heeft wat gezien? Wanneer? Vanaf welk IP?
- **Je kunt risico's communiceren aan niet-technici**
 - Je weet het verschil tussen een CVSS-score en wat het betekent voor een patiënt in een ziekenhuisbed.
- **Je weet dat "het werkt" niet hetzelfde is als "het is veilig"**
 - Software kan functioneel correct zijn en tegelijk een kritiek beveiligingsrisico vormen. Beide dimensies zijn jouw verantwoordelijkheid.



